

EVM – Volunteer Event Management System

(Consolidated Documentation)

This document consolidates the mandatory scope delivered for the EVM (Volunteer Event Management System) built on OutSystems. It outlines completed features, the layered architecture, key entities, the reminder timer design, a test checklist, and a backlog of enhancements.

1. Features Completed (Mandatory)

Volunteer Web App:

- Browse Upcoming Events (Title, Date/Time, Location).
- Event Details with live weather (OpenWeather) for the event's coordinates.
- Sign up for events with capacity enforcement (MaxVolunteers) and duplicate prevention.
- My Events: Upcoming & Past.
- Submit Feedback after the event (rating + comments; visible only when eligible).

Admin Web App:

- CRUD Events (Title, Description, Location, Lat/Lng, Start/End, MaxVolunteers, Status).
- Location search via OpenWeather Geocoding to auto-fill Location + Lat/Lng.
- CRUD Email Templates with tokenized Subject/Body (Key = "ReminderBeforeStart").
- Reminder Timer to notify volunteers X days before start; NotificationLog for idempotency.

Weather Integration:

- OpenWeather Current Weather by lat/long; Direct Geocoding (search by text).

Email & SMTP:

- Gmail SMTP (Service Center) using App Password; plain-text reminders sent from Admin Timer.

2. Architecture (Layers & Modules)

The implementation follows the OutSystems Architecture Canvas: End-User Apps → Core → Foundation. UI modules never call external providers directly; Core encapsulates business logic and integrations.

2.1 Modules

1. Foundation

- EVM_Foundation_OpenWeather – REST to OpenWeather (Direct Geocoding, Current Weather).

2. Core

- EVM_Core_Events – Event & EventStatus; weather accessors; optional WeatherCache; helpers for event operations.
- EVM_Core_Participation – Participation & ParticipationStatus; IsUserRegistered, GetCapacityRemaining, SignUpForEvent.
- EVM_Core_Feedback – Feedback; CanSubmitFeedback (ended or Completed + registered + not yet given), SubmitFeedback.
- EVM_Core_Notifications – EmailTemplate, NotificationLog, Status; GetEmailTemplateByKey, ApplyTokens, BuildTokensForEventUser.

3. End-User Apps

- EVM_Admin_Web – Event & Email Template admin, and EventReminder_Run Timer.
- EVM_App_VolunteerWeb – Upcoming Events list, Event Details (with weather), My Events (Upcoming/Past), Feedback popup.

3. Data Model (Entities)

Entity	Key Attributes	Notes
Event	Title, Description, Location, Latitude, Longitude, Start, End, MaxVolunteers, StatusId	Status: Draft/Published/Completed.
EventStatus (SE)	Draft, Published, Completed	Static entity.
Participation	EventId, UserId, StatusId, RegisteredAt, CancelledAt	Unique (EventId, UserId).
ParticipationStatus (SE)	Registered, Cancelled	Static entity.
Feedback	EventId, UserId, Rating,	Unique (EventId, UserId).

	Comments, CreatedAt	
EmailTemplate	Key, Name, SubjectTemplate, BodyTemplate, IsActive, UpdatedAt, UpdatedBy	Tokenized templates; Key unique.
NotificationLog	Type, SendAt, StatusId, ErrorMsg, EmailTemplateId, RegistrationId	Idempotent reminder tracking.
Status (SE)	Pending, Sent, Failed	Used by NotificationLog.
(Optional) WeatherCache	Lat, Lng, ProviderHash, Json, CachedAt	If caching is enabled in Core.

4. Reminder Timer (EventReminder_Run)

Runs every 15–30 minutes. For events whose Start falls within a configurable window at Now + N days, the timer sends a reminder email to each registered volunteer who hasn't already been sent one, and writes a NotificationLog row (Sent/Failed).

- Key configuration (Site Properties in Admin Web):
 - ReminderDaysBefore
 - ReminderWindowMinutes
 - MaxEmailsPerRun
- Template Key: "ReminderBeforeStart"; tokens supplied by BuildTokensForEventUser.

5. Test Checklist

- Create a Published event with Start = Now + N days + a few minutes; set Lat/Lng.
- Register a volunteer; verify capacity decrements (unless unlimited).
- Manually run EventReminder_Run in Service Center; confirm email received; check NotificationLog.
- After event end, submit feedback; verify one feedback per user/event.

6. Enhancements (If Time Permits)

- Optional tasks:
 - Bootstrap data
 - Mobile app (check-in, Camera upload).
 - BPT post-event process: follow-ups, feedback collection, participation report.

- Offline check-in/notes with Local Storage & sync.
- Custom Extension: distance between coordinates; participation code; sanitizers.
- Operational & UX:
 - LogMessage helper (centralized structured logs, level toggle).
 - Google Maps: map + autocomplete; compute distances; sort events near volunteer.
 - Event images: attach & include in emails. If S3 disallowed, expose an API that streams image bytes with MIME type for safe linking in emails.
 - Invite volunteers by entering emails and sending invitations with a deep link.
 - Anonymous participation mode (no Users dependency), with optional email confirmation/OTP.
 - tatistics dashboard: events vs attendance, feedback averages, registrations over time, revenue.

Document generated for the EVM project – OutSystems assessment.

Appendix A — Architecture Canvas (OutSystems)

This section documents the solution using the OutSystems Architecture Canvas. It clarifies module boundaries, data ownership, dependencies, and the allowed direction of references.

A.1 Layers & Allowed Dependencies

End-User Apps → Core → Foundation → External Services

Rules: End-User Apps depend only on Core; Core depends on Foundation; Foundation talks to external services. No UI→Foundation or Foundation→Core references. Entities are owned by Core modules.

A.2 End-User Apps

Module	Responsibilities	Key Screens/Timers
EVM_App_VolunteerWeb	Volunteer UI; browse events, event details, sign up, My Events (Upcoming/Past), submit feedback; shows weather via Core.	Upcoming Events, Event Details (Weather Banner), My Events, Feedback Popup
EVM_Admin_Web	Admin UI & batch jobs; manage Events and Email Templates; drives the reminder Timer using Core logic.	Admin Events Editor, Email Templates, Timer: EventReminder_Run

A.3 Core

Owns business entities and rules. Exposes server actions used by UI. May implement caching. No direct external calls except via Foundation.

Module	Owns Entities	Exposed Actions (examples)	Notes
EVM_Core_Event	Event, EventStatus, Participation, ParticipationStatus	IsUserRegistered, GetCapacityRemaining, SignUpForEvent, GetWeatherByCoords/ForEvent (wraps Foundation)	Enforces capacity, status windows; acts as façade for weather.
EVM_Core_Feedback	Feedback	CanSubmitFeedback, SubmitFeedback	One feedback per user/event; gate by (End <

			Now OR Completed) + Registered
EVM_Core_Notifications	EmailTemplate, NotificationLog, Status	GetEmailTemplateByKey, Apply/ReplaceTokens, BuildTokensForEventUser	Templates are tokenized; NotificationLog provides idempotency

A.4 Foundation

Stateless integrations with external systems. No business rules, no ownership of core data.

Module	Integration	Endpoints Used
EVM_Foundation_OpenWeather	OpenWeatherMap APIs	Direct Geocoding /geo/1.0/direct; Current Weather /data/2.5/weather (units=metric)

A.5 External Services

- OpenWeatherMap (Geocoding + Current Weather)
- Gmail SMTP (outbound email via Service Center settings)

A.6 Data Ownership & Delete Rules

Entity	Owner Module	References (Delete Rule)	Notes
Event	EVM_Core_Event	—	Status: Draft/Published/Completed
Participation	EVM_Core_Event	Event(Delete), User(Protect), Status(Protect)	Unique (EventId,UserId)
Feedback	EVM_Core_Feedback	Event(Delete), User(Protect)	Unique (EventId,UserId)
EmailTemplate	EVM_Core_Notifications	—	Key unique
NotificationLog	EVM_Core_Notifications	Registration(Ignore), EmailTemplate(Ignore)	Audit & idempotency

g

ns

e), Status(Protect)

A.7 Cross-Cutting Concerns

- Site Properties: ReminderDaysBefore, ReminderWindowMinutes, MaxEmailsPerRun, OpenWeatherApiKey, VolunteerPortalBaseUrl
- Security & Roles: Registered users (Volunteers); Admin role for admin module.
- Timer: EventReminder_Run (Admin Web) – sends reminders and writes NotificationLog (idempotent).
- Caching: WeatherCache (optional) in Core if desired for performance.

A.8 UI Composition & Reuse

- WB_WeatherBannerByCoords: reusable weather banner block bound to Lat/Lng (calls Core).
- WB_FeedbackDialog: reusable popup for submitting post-event feedback.

Appendix B — Architecture Canvas Diagram

The following diagram illustrates the Architecture Canvas layout used in this solution:

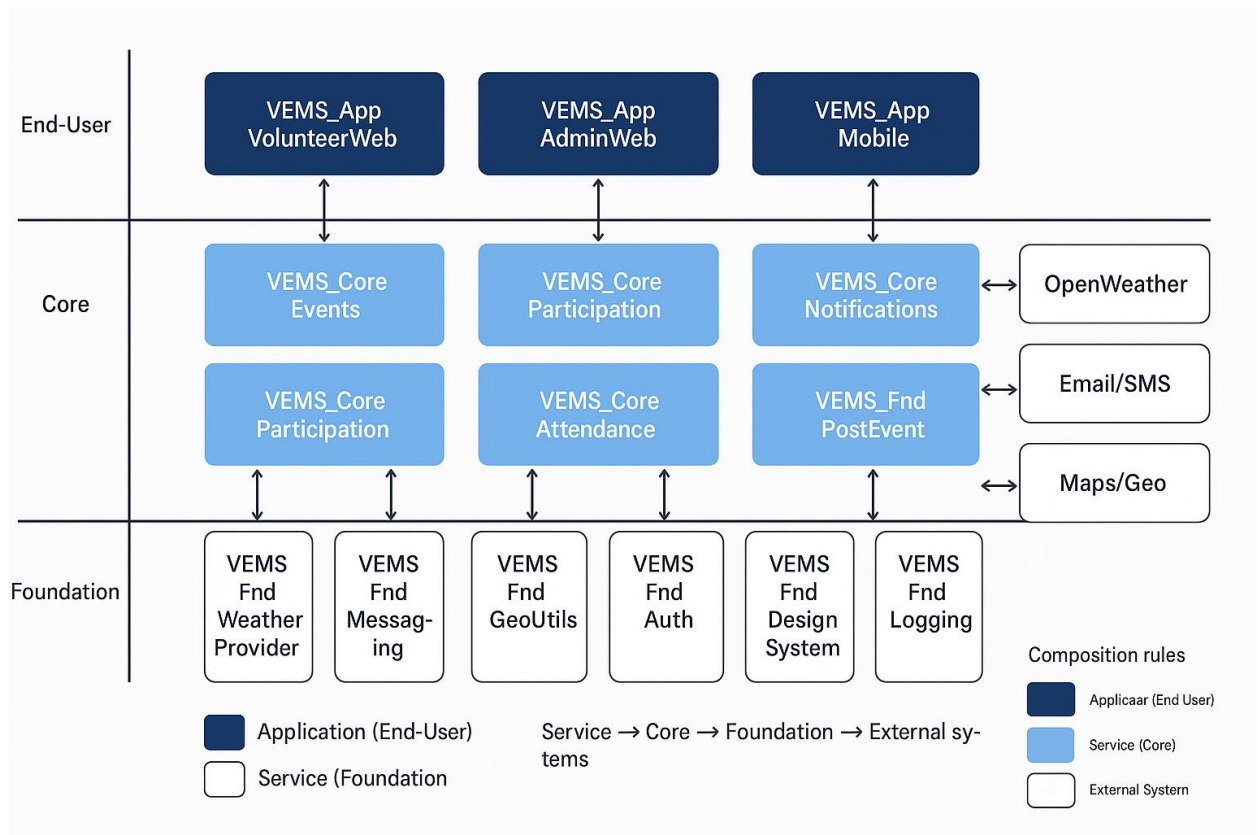
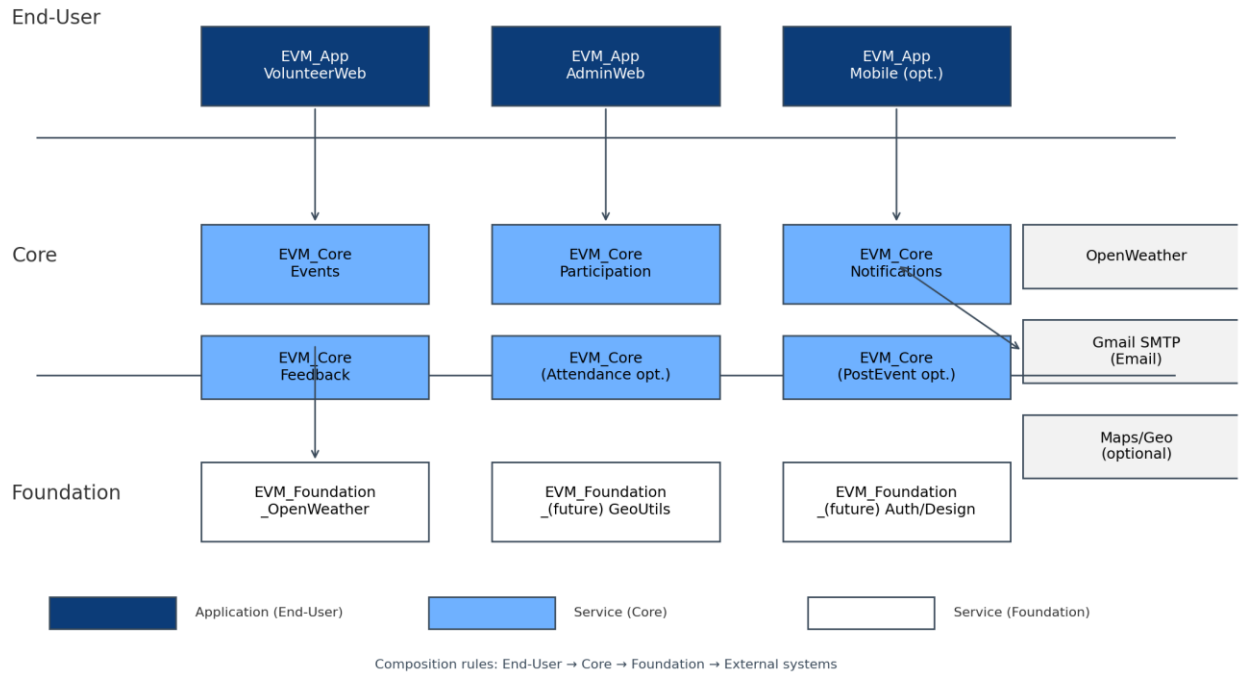


Figure B-1. Architecture Canvas showing End-User Apps, Core modules, and Foundation services.

Appendix B — Architecture Canvas Diagram (EVM naming)

Updated diagram reflecting module names prefixed with EVM_.



Appendix C — Updated Architecture (Consolidated Core)

Note: Participation and Feedback have been consolidated into a single module, EVM_Core_Events. This module now owns entities Event, Participation, and Feedback, and exposes all related actions (SignUpForEvent, GetCapacityRemaining, CanSubmitFeedback, SubmitFeedback). EVM_Core_Notifications remains separate to own EmailTemplate and NotificationLog.

